i am doing research in computer science and I am not familiar with different types of metrics and mathematical notations help me to understand them when i ask you simplify them so that i can understand the meaning and when and where to use them and any example that might be easy to understand and useful.

Yes — I can do that, and I'll explain each metric or notation in very simple language with meaning, when to use it, where it appears, and an easy example. Your request fits well with a beginner-friendly style because mathematical notation in computer science often acts like a compact shorthand for ideas such as performance, error, probability, or optimization.[1]

How I'll explain it

In computer science research, metrics are numbers used to judge how good a model, system, or algorithm is, while notation is the symbolic language papers use to describe data, formulas, and procedures. Common metric families include classification metrics, regression metrics, ranking metrics, and system performance metrics, and the right choice depends on the task and the kind of mistakes that matter.[2] [3]

When you ask me about something, I can break it into four parts:

- What it means in plain English.
- When to use it.[3]
- Where it is commonly used, such as ML, systems, networking, or research papers.[2] [3]
- One small example with no heavy math unless you want it.[1]

Common notation

A lot of research notation repeats across papers, even when topics change. Scalars are usually single numbers like x, vectors are ordered lists often written in bold such as $\mathbf{x}$, matrices are tables of numbers often written with capital letters like X, and indices like x_i usually mean "the i-th item."[4] [1]

You will also often see:

- $\sum$: "add many things together."[1]

- $\in$: "is an element of" or "belongs to a set."[1]
- $\mathcal{D}$: dataset or data distribution, depending on context.[5]
- θ: model parameters, such as weights a model learns.[4]

Example: if a paper writes x_i, it usually means "sample number i." If it writes $\sum_{i=1}^{n} x_i$, it means "add all samples from 1 to n."[1]

Common metrics

For classification, the most common metrics are accuracy, precision, recall, specificity, and F1 score. Accuracy is the fraction of all predictions that were correct, precision tells you how trustworthy the positive predictions are, recall tells you how many real positives were found, and F1 balances precision and recall.[6] [7] [3]

A simple example is spam detection:

- Accuracy: "How many emails did the model label correctly overall?"[8] [6]
- Precision: "Of the emails marked spam, how many were actually spam?"[7] [6]
- Recall: "Of all real spam emails, how many did the model catch?"[8] [6]
- F1: useful when both false alarms and missed spam matter.[6] [7]

For regression, common metrics are MAE, MSE, and R^2. MAE measures average absolute error, MSE punishes larger errors more strongly because it squares them, and R^2 measures how much of the variation in the target is explained by the model.[3]

When to use which

Accuracy can be misleading when classes are imbalanced, such as fraud detection or anomaly detection, because a model can look good by mostly predicting the majority class. In those cases, precision, recall, F1, Matthews correlation coefficient, or confusion-matrix-based analysis are usually more informative.[6] [3]

A practical rule is:

- Use accuracy when classes are balanced and all mistakes matter similarly.[8] [3]
- Use precision when false positives are costly, such as wrongly flagging normal traces as anomalies.[3] [6]
- Use recall when missing a real positive is costly, such as missing an attack or anomaly.[8] [6]
- Use F1 when you want a single score balancing precision and recall.[7] [6]
- Use MAE or MSE for continuous-value prediction tasks.[3]

How to ask me

You can send me a symbol, formula, or metric name, and I'll simplify it in a fixed format that is easy to study. Based on your preference for very simple, topic-by-topic explanations, I can keep each explanation minimal and intuitive rather than math-heavy.

Use prompts like:

- "Explain precision in simple words."
- "What does x_i mean in a paper?"
- "When should I use F1 instead of accuracy?"
- "Simplify this formula: $L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$."
- "Explain this notation from my research paper line by line."

Start with one item, such as $\mathbf{accuracy}$, precision, recall, F1, MAE, MSE, x_i, $\sum$, $\arg\max$, $P(A \mid B)$, or θ. Which one should we do first?

⁂

I do not about entropy help me to understand it.

Entropy here means "how **uncertain** something is," and cross-entropy means "how wrong your predicted probabilities are compared with what actually happened." In your screenshot, the model predicts the next event in a trace, and the loss becomes small when it gives high probability to the true next event and large when it is surprised by that event.[16]

Entropy idea

Think of entropy as the amount of surprise in a system. If one event is almost always the next one, uncertainty is low; if many different next events are all possible, uncertainty is high.[16]

Easy example:

- If a system always goes $A \to B$, then the next event is easy to guess, so entropy is low.[16]
- If after A, the next event could be B, C, D, or E with similar chance, then entropy is higher because the future is less predictable.[16]

What cross-entropy means

The paper is not mainly measuring entropy by itself; it is using **cross-entropy loss** to compare two things: the true outcome and the model's predicted probabilities. The formula $H(p, q) = -\sum_{e \in E} p(e) \log(q(e))$ says: "look at the true distribution p, check what probability the model q gave to those true outcomes, and assign a larger penalty when the model gave them low probability."[16]

In plain words:

- p: what is actually correct.[16]
- q: what the model predicted.[16]
- Loss small: model was confident about the correct event.[16]
- Loss large: model gave low probability to the correct event.[16]

Why the simple formula appears

For next-event classification, the true answer is usually just one correct event, so the paper says p is one-hot encoded. That means one event has value 1 and all others have value 0, so the long summation simplifies to $H(p, q) = -\log(q_{\text{true}})$.[16]

That simpler formula means you only care about the probability assigned to the actual next event:

- If the model says the true event has probability 0.9, the loss is low.[16]
- If the model says the true event has probability 0.1, the loss is high.[16]

Very simple example

Suppose the true next event is "disk read." The model predicts:

- disk read = 0.8
- network call = 0.1
- mutex lock = 0.1 [16]

This is good because the true event got high probability, so cross-entropy is low. But if the model predicts:

- disk read = 0.05
- network call = 0.7
- mutex lock = 0.25 [16]

then the model is very wrong about the real next event, so cross-entropy becomes high. That is why the paper says high sequence loss can signal anomaly or novelty: the observed behavior does not match what the model expected from normal traces.[16]

Sequence loss

The last formula averages this penalty over all events in the sequence: $H_{\text{sequence}} = \frac{1}{n}\sum_{j=1}^{n} H(p_j, q_j) = -\frac{1}{n}\sum_{j=1}^{n} \log(q_{\text{true}}(j))$. This means the model checks each step in the trace, measures how surprised it is, and then averages that surprise over the full sequence.[16]

So the interpretation is:

- Low average loss: the sequence looks normal to the model.[16]
- High average loss: the sequence looks unusual, unexpected, or anomalous.[16]

Here is the shortest intuition to remember: entropy = uncertainty, cross-entropy = how surprised your model is by the truth, and high cross-entropy in tracing means "this behavior did not look like normal behavior."[16]

Would you like me to explain the symbols in the formula one by one, like p, q, $\sum$, log, and one-hot encoding?

⁂

3.2. Event duration modeling Modeling the duration of kernel events enables the detection of perfor mance variations that may not manifest in the event types or their order. In many cases, a system under stress may trigger the same events in the same sequence, but with increased latency. Therefore, modeling only the event types may fail to capture such changes. By incorporating duration informa tion, we aim to improve anomaly detection sensitivity and broaden the range of detectable behaviors. We treat duration modeling as a classification task by discretizing du rations into ordinal categories, avoiding the complexities of regression while retaining sensitivity to significant timing shifts. Section 4 details the binning process. Similar to event sequence modeling, we use common event arguments as input features, since durations are often highly correlated with these at tributes. However, the model does not need to predict durations for all events in the trace; instead, it focuses only on events that signal the end of an execution interval. The primary training objective is to predict the duration of an event's execution. As shown in Figure 2, this duration is specified by two distinct events in the trace: the 'Entry' (denoted as eentry i eexit i) and 'Exit' (denoted as) events, which are particularly relevant for system call events. While the model encounters an 'Entry' event eentry i , no prediction is expected. The predicted durations are only considered for loss calculation when the

interval is closed, which means when an ‘Exit’ event eexit i 11 is issued.

This section means the paper is trying to detect anomalies not only from **what** events happen, but also from **how long** they take. A system may show the same event order as normal behavior, yet still be abnormal because some operations become much slower, so the paper models duration as an extra signal for anomaly detection.[17]

Main idea

The authors are saying that event type alone is sometimes not enough. For example, a stressed system may still produce the same sequence of kernel events, but the latency of those events may increase, and that timing change can reveal a problem.[17]

So they add duration modeling:

- Event sequence modeling asks: “What event comes next?”[17]
- Duration modeling asks: “How long did this event take?”[17]

Why classification

Instead of predicting an exact time value, they convert durations into ordered groups such as short, medium, and long. They do this because classification is often easier and more stable than regression, while still allowing the model to detect important timing shifts.[17]

So if raw durations were:

- 2 microseconds
- 8 microseconds
- 20 microseconds[17]

they may turn them into categories like:

- Short
- Medium
- Long[17]

Entry and exit events

The duration of an execution interval is defined by two trace events: an Entry event and its matching Exit event. The paper says the model sees the Entry event first, but it only computes prediction loss when the Exit event appears, because only then is the full duration known.[17]

Simple example:

1. A system call starts at Entry.[17]
2. Time passes while the call executes.[17]
3. The system call ends at Exit.[17]

4. Duration = Exit time minus Entry time.[17]

So Entry opens the interval, and Exit closes it.[17]

Equation 5

The formula $L(p', q') = -\sum_{d \in D} p'(d) \log(q'(d))$ is the cross-entropy loss for duration categories. It has the same basic meaning as before: compare the true duration class with the probabilities predicted by the model.[17]

In simple words:

- D: the set of possible duration categories, such as short, medium, long.[17]
- $p'(d)$: the true answer.[17]
- $q'(d)$: the model's predicted probability for category d.[17]

If the true duration is "long" and the model gives high probability to "long," the loss is low. If the model gives low probability to "long," the loss is high.[17]

Ordinal encoding

The paper says duration categories are ordinal, which means they have a natural order. Short, medium, and long are not just different labels; they are arranged from smaller to larger.[17]

That is why the paper uses ordinal binary encoding instead of plain one-hot encoding. In the example with three categories, it encodes:

- d_1 shortest as 00.[17]
- d_2 medium as 10.[17]
- d_3 longest as 11.[17]

This encoding tells the model that medium is closer to long than short is to long. That helps the loss reflect severity, so predicting short instead of long is treated as a worse mistake than predicting medium instead of long.[17]

Equation 7 and 8

Equation 7 applies binary cross-entropy to each bit of that ordinal code. Instead of predicting one label directly, the model predicts each bit and gets penalized for each incorrect bit.[17]

In plain language:

- y_j: the true bit value at position j.[17]
- $\hat{y}_j$: the model's predicted probability that this bit should be 1.[17]
- L_{ordinal}: the total loss across all bits.[17]

Equation 8 then averages that ordinal loss over all relevant exit events in the sequence. So the final sequence loss says, on average, how wrong the model was when predicting event durations for completed intervals.[17]

A very easy way to remember it is:

- Event model = "Did the right type happen next?"[17]
- Duration model = "Did it take the expected amount of time?"[17]
- Higher loss = behavior looks less normal.[17]

Would you like me to next explain Equation 7 term by term, especially what $y_j \log(\hat{y}_j) + (1 - y_j) \log(1 - \hat{y}_j)$ really means in simple words?

⁂

3.3. Multi-task Learning Given that both the event model and the duration model benefit from the same set of features derived from trace data, as well as the correlation between events and their durations, we modify the architecture of the neu ral network to implement a multi-task setup. Multi-task learning allows the model to leverage the shared knowledge gained from each task. By unifying these models, we provide the network with a holistic view of both aspects of the trace. To accommodate both tasks, the encoder of the network is shared, while each task has a separate fully connected layer on top of the encoder to produce the appropriate output. Similar to the approach described in [33], the aggregated loss function used during training is calculated using Equa tion 9. The hyperparameter λ regulates the contribution of each modeling task during backpropagation. LTotal = λLEvent + (1 − λ)LDuration (9) As mentioned, the calculated loss for each sequence is the key to determin ing its normality. With two loss values from the event and duration models during the inference phase, we need a combined scheme to make a prediction utilizing both models. However, there are different scales for the loss values of each model. To address this discrepancy and enable harmonious integra tion of the two models, the loss values of each model are normalized using the Median Absolute Deviation (MAD). This approach is chosen specifically for its robustness against outliers and points in the extreme tails of the distri bution [34].

For a set of loss values L = {l1,l2,...,ln}, resulted from a single model, the normalized values are computed by Equation 10. 13 l′ i = li − median(L) (10) MAD(L) where MAD(L) = median(|li −median(L)|). The normalized event and duration losses for each sequence are then summed to yield a final anomaly score. This composite score enables a holistic assessment of system deviations by incorporating both control flow (event type) and timing behavior (duration).

This section says the paper combines two learning goals in one network: predicting the next event type and predicting event duration. The idea is that both tasks come from the same trace data, so learning them together can help the model build a better shared understanding of system behavior.[18]

Multi-task learning

Multi-task learning means one model learns several related tasks at the same time instead of training completely separate models. In this paper, the shared encoder learns common trace patterns, while two separate output layers, often called heads, produce task-specific predictions for event type and duration.[18]

A simple analogy is one student studying both grammar and writing from the same language lessons. The basic knowledge is shared, but the final answers for each task are different.[18]

Equation 9

The formula $L_{\text{Total}} = \lambda L_{\text{Event}} + (1 - \lambda) L_{\text{Duration}}$ means the total training loss is a weighted combination of the two task losses. The parameter λ controls how much importance the model gives to the event task versus the duration task during training.[19] [18]

Plain meaning:

- L_{Event}: how wrong the model is on next-event prediction.[18]
- L_{Duration}: how wrong the model is on duration prediction.[18]
- λ: the balance knob between the two.[19] [18]

Example:

- If $\lambda = 0.8$, the event loss gets more weight.[19] [18]
- If $\lambda = 0.5$, both tasks matter equally.[19] [18]
- If $\lambda = 0.2$, the duration loss matters more.[19] [18]

Why normalize losses

During inference, the paper gets two loss values for each sequence, one from the event model and one from the duration model. The problem is that these losses may have different numeric scales, so adding them directly could let the larger-scale loss dominate even if it is not more important.[20]

That is why the paper normalizes each model's losses before combining them. The goal is to put both losses on a more comparable scale so the final anomaly score uses both control-flow information and timing information fairly.[20]

MAD normalization

The paper uses Median Absolute Deviation, or MAD, because it is robust to outliers. MAD is based on the median and the median of absolute distances from that median, so a few extreme values do not distort it as much as standard-deviation-based scaling would.[21] [22]

The formula $l'_i = \frac{l_i - \text{median}(L)}{\text{MAD}(L)}$ means:

- Start with one loss value l_i.[22]
- Subtract the typical center value, which is the median of all losses.[22]
- Divide by MAD, which measures typical spread in a robust way.[21] [22]

So the normalized value tells you how far a sequence's loss is from the usual loss level for that model, measured in a way that is less sensitive to extreme cases.[21] [22]

Final anomaly score

After normalization, the paper adds the normalized event loss and normalized duration loss to get one final anomaly score. A high final score means the sequence looks unusual in terms of event order, duration behavior, or both.[23]

This is why the authors call it a holistic assessment: one part checks control flow, and the other checks timing behavior. If a trace has normal event order but abnormal latency, the duration part can still raise the final anomaly score.[23]

Here is the simplest way to remember the whole section:

- Shared encoder = one common brain for both tasks.[18]
- Two heads = one for event type, one for duration.[18]
- Weighted total loss = train both tasks together.[19] [18]
- MAD normalization = make the two loss scales comparable.[22] [21]
- Final anomaly score = combine both views of abnormality.[23]

Would you like me to explain Equation 10 with a tiny numeric example, using 4 or 5 loss values step by step?

⁂

3.4. Root cause Analysis Whenanovelty is detected in the system trace, it is essential to identify its root cause and ensure transparency in the model's decision-making process. This section presents a root cause analysis method that operates on top of the trained trace model and does not require further training. This module guides adaptive tracing by enabling faster and more informed responses to emerging issues. Upon detecting an abnormal request trace, the events that led the model to classify it as abnormal often contain valuable signals for diagnosing the deviation. As discussed earlier, abnormal traces deviate significantly from model predictions, meaning that certain events are assigned low probabilities. These events—whether incorrectly predicted or entirely unexpected—can be analyzed to locate the source of deviation. Our method extracts these events and uses them to identify the root cause of each abnormal trace. Figure 3 illustrates the distribution of incorrect predictions in the OPCache anomaly set. Each type of anomaly exhibits a unique deviation pattern, reinforcing the viability of using error patterns for root cause inference. Figure 4 compares distributions of incorrect predictions across various out-of-distribution (OOD) sets for frequently occurring events, showing how the deviation profiles differ across anomaly types. As outlined in Algorithm 1, after the model generates its predicted se quence, either events or durations, the count errors function is invoked to compute the error count for

each event type. This produces an error vector for the sequence, which is later used for clustering and classification. The behavior of count errors depends on the model used:

• Event Model: Compares each event in the predicted sequence ˆS with the ground truth S and
• Duration Model: Examines each event's predicted duration against the actual duration, aggregating the count of deviations.
• Combined Model: Computes errors for both events and durations, as described above, and aggregates them by averaging to produce a unified error.

Once the error vector is formed, it is compared against the centroids of pre-labeled error vector clusters using cosine similarity. The label of the clos est cluster centroid is assigned as the predicted root cause. Pre-clustering the labeled vectors not only improves efficiency by avoiding repeated com putation during inference but also mitigates noise by capturing stable error patterns in each anomaly type.counts mismatches.

This section says that after the model detects an abnormal trace, it tries to explain **why** by looking at which events or durations were predicted badly. The main idea is that different anomaly types create different error patterns, and those patterns can be used like signatures to infer the root cause.[33]

Main idea

The paper does not stop at saying "this trace is abnormal." It also builds a root-cause module that looks at the model's mistakes and uses them to guess which known anomaly type caused the deviation.[33]

So the logic is:

- Normal trace: predictions and reality are close.[33]
- Abnormal trace: some events or durations get low predicted probability or clear mismatches.[33]
- Those mismatches become clues for diagnosis.[33]

Error vector

The key object here is the error vector. An error vector is just a compact summary showing how many mistakes happened for each event type, or for durations, in one sequence.[33]

Very simple example:

- read: 5 errors
- write: 1 error
- open: 0 errors
- close: 3 errors[33]

That list of counts becomes the fingerprint of the abnormal trace. The paper says this vector is built differently depending on the model, but in all cases it represents where the model observed deviations.[33]

Three model cases

The paper describes three ways to count errors:

- Event model: compare predicted events with the true event sequence and count mismatches.[33]
- Duration model: compare predicted duration category with the true duration category and count deviations.[33]
- Combined model: compute both kinds of errors and merge them, here by averaging, to produce one unified error representation.[33]

So the combined model uses both control-flow mistakes and timing mistakes. That makes sense because some anomalies mainly change event order, while others mainly change latency.[33]

Clustering idea

Once the error vector is formed, the paper compares it to cluster centroids built from labeled anomaly examples. A centroid is the representative center of a group, so each anomaly type has a typical error-pattern center.[33]

This means the paper first learns typical patterns such as:

- anomaly type A often causes many read errors;[33]
- anomaly type B often causes duration deviations in exit events;[33]
- anomaly type C affects a different set of event types.[33]

Then for a new abnormal trace, it asks: "Which known pattern is this error vector most similar to?"[33]

Cosine similarity

The paper uses cosine similarity to compare the new error vector with cluster centroids. Cosine similarity measures how similar two vectors are in direction, which is useful when you care more about the pattern of errors than just the total number of errors.[33]

Simple intuition:

- If two traces both mostly fail on the same event types, their vectors point in a similar direction, so cosine similarity is high.[33]
- If they fail on very different event types, the vectors point in different directions, so cosine similarity is low.[33]

That is why cosine similarity is a reasonable choice for root cause inference from error patterns. It focuses on the shape of the mistake profile.[33]

Why clustering helps

The paper says pre-clustering labeled error vectors improves efficiency and reduces noise. Instead of comparing a new abnormal trace with every old training example, the system compares it with a smaller number of centroids, which is faster and more stable.[33]

So the full process is:

1. Detect an abnormal trace using loss.[33]
2. Extract mistakes from events, durations, or both.[33]
3. Build an error vector.[33]
4. Compare that vector with anomaly centroids using cosine similarity.[33]
5. Assign the label of the closest centroid as the predicted root cause.[33]

The easiest way to remember this section is: anomaly detection says “something is wrong,” and root cause analysis says “the pattern of prediction mistakes suggests what kind of problem it is.”[33]

Would you like me to explain Algorithm 1 line by line in very simple words next?

⁂

4. Data Collection To evaluate the effectiveness of the proposed LMAT approach, we require kernel traces from a real-world application that satisfy three key criteria: (1) sufficient scale to support training of deep neural networks, (2) detailed ker nel event arguments to compute event duration and support effective model ing, and (3) coverage of both normal and anomalous system behaviors. The dataset introduced by [12] is the most suitable publicly available resource for 16 Table 2: Specification of the system used to collect trace data for the experiments. Name CPU Specification Intel® CoreTM i7 3.60GHz Memory 2*16 GB of DDR5 configured at 4400 MHz Hard disk 1TB DT01ACA100 Toshiba Desktop OS Linux Ubuntu 22.04.2 LTS this purpose. It includes traces of two million web requests with correspond ing event arguments, encompassing both normal operations and scenarios involving known system issues. While part of our evaluation is based on this dataset to enable direct comparison with prior work, we also collect an additional dataset of similar scale using the same instrumentation methodology. This enables us to collect traces of other types of system behavior, which present greater challenges to trace models and potentially address the gaps left by the previous dataset. Our objective is to enhance the dataset's diversity, thereby providing a more accurate representation of real-world applications. In particular, we focus on performance-related issues that significantly impact the duration of event processing within the

system. Following the methodology outlined in [12], we established a test environ ment consisting of an Apache2 web server and a client executing a benchmark to send HTTP requests to the server. The experiments were conducted on a machine equipped with an Intel® CoreTM i7 CPU and 32 GB of DDR5 RAM,specifications that are robust and detailed in Table 2. A clean installa tion of Linux Ubuntu 22.04.2 LTS ran on the system, providing a stable and controlled environment for the tests. On the client side, the wrk2 benchmark was configured to send 1000 requests per second to fully engage the server. We collected traces exclusively from the server side, as it is the principal source of latency within the system. The web server was traced for 1,000 seconds to compile the training set and for 100 seconds each to assemble both the validation and test sets under normal and noisy conditions, result ing in 1,000,000 requests for the training set and 100,000 requests for each of the test and validation sets. The noisy conditions correspond to various types of intentionally injected anomalies that simulate realistic system-level misconfigurations and performance bottlenecks.

This section explains how they built the dataset used to test the method. In simple terms, they needed a large set of real kernel traces, with enough detail to model both event order and event duration, and with both normal behavior and injected abnormal behavior.[34] [35]

What they needed

The paper says the trace data had to satisfy three conditions: it had to be large enough for deep learning, include detailed kernel event arguments, and contain both normal and anomalous behavior. That is because the model needs many examples to learn patterns, enough trace detail to compute durations, and abnormal cases to test whether anomaly detection actually works.[35] [34]

So the dataset is not just “a lot of traces.” It must also contain the right kind of information for the two tasks discussed earlier, event prediction and duration prediction.[34]

Why collect more data

They used an existing public dataset so they could compare against earlier work, but they also collected a new dataset of similar scale. The reason is that one dataset may not cover enough behavior types, especially performance-related problems that mainly change timing rather than event order.[34]

This is important because a realistic anomaly detector should work on varied system behaviors, not just on one narrow benchmark case. A more diverse dataset gives a better picture of whether the model generalizes.[34]

Experimental setup

Their test environment had an Apache2 web server and a client that sent HTTP requests using wrk2. wrk2 is an HTTP benchmarking tool designed to generate a constant throughput load, and it supports a target request rate through the $-R$ or $--rate$ option.[36] [37]

The paper says the client sent 1000 requests per second to keep the server busy. The server ran on Ubuntu 22.04.2 LTS, and they traced only the server side because that is where they expected the main latency behavior to appear.[36] [34]

Train, validation, test

The paper split data into training, validation, and test sets, which is standard in machine learning. The training set is used to learn model parameters, the validation set is used to tune choices and monitor performance during development, and the test set is used for the final unbiased evaluation.[35] [34]

The durations in the section mean:

- Training traces: 1000 seconds. [query context]
- Validation traces: 100 seconds. [query context]
- Test traces: 100 seconds. [query context]

And the request counts mean:

- 1,000,000 requests for training. [query context]
- 100,000 requests for validation. [query context]
- 100,000 requests for testing. [query context]

Noisy conditions

The “noisy conditions” are the intentionally injected anomalies. These simulate realistic system problems or bottlenecks so the researchers can check whether the model can distinguish normal traces from problematic ones.[34]

So the full idea is:

- Normal traces teach the model what healthy behavior looks like.[35]
- Noisy traces test whether the model notices deviations.[34]
- Detailed kernel traces make it possible to study both event flow and timing. [query context]

One small wording note: several details in your pasted paragraph, such as the exact hardware table, the exact time windows, and the exact request counts, come from the paper text you shared rather than from an external source, so I treated those as part of the provided context.[34]

Would you like me to convert this whole Section 4 into a very short “easy notes” version you can use for studying?

⁂

4.2. Duration Preprocessing Labeling the events in trace data with their respective durations involves identifying the start and end kernel events that define an operation interval within the system. The interval begins with the timestamp of an 'Entry' event and concludes with the timestamp of the corresponding 'Exit' event. These 'Entry' and 'Exit' events are matched based on their PID and TID arguments. The duration of a specific interval is calculated by determining the difference between the timestamps of the matched 'Entry' and 'Exit' events, and this duration is then tagged to the 'Exit' event. Conversely, the 'Entry' event is assigned a 'none' tag for its duration, indicating that no duration is measured at that point. This method ensures that each operation interval is accurately represented by its corresponding duration, which is crucial for precise system analysis and performance metric evaluation. In the following section, we describe how these durations are categorized and adjusted throughout system operation. 4.2.1. Duration Categorization We adopted a custom percentile-decreasing binning strategy tailored to the specific characteristics of our data. In Linux kernel events, the major ity of events are completed quickly, with slower events being a minority, as illustrated in Figure 5. Therefore, we aim to preserve this pattern after cate gorizing event durations. To achieve this, our method assigns most durations to the category representing the fastest events, while progressively decreasing the number of events in

categories with longer durations. The duration of each event is categorized into one of the k categories. Given a set of durations for a specific type of event, we first sort them in decreasing order. We then divide the durations into bins such that the num ber of data points in each bin gradually decreases. The distribution of data points across bins is calculated based on a linearly decreasing function. This function starts at k, where k is the total number of bins, and decreases by 1 with each subsequent bin. The resulting fractions from this function are then normalized so that their sum equals 1. This approach can be formalized as follows: Given k bins, the total number of points is the sum of an arithmetic series: 19 Duration Distribution with Categories for syscall_exit_recvfrom 4000 3000 Frequency 2000 1000 0 1000 2000 3000 4000 5000 Duration (ns) 6000 7000 8000 Figure 5: Distribution of "recvfrom" system call durations binned into three categories. Total = k ·(k +1) 2 The fraction of points in each bin, starting from the first bin (indexed with i=1) to the nth bin, is calculated as follows: f(i) = k −i+1 Total This yields a decreasing sequence of fractions that sum to 1, ensuring that all points are allocated across the bins. This binning strategy allows us to construct bins that better reflect the actual distribution of durations in our dataset. Moreover, our method miti gates the influence of potential outliers or exceptionally long durations, which could disproportionately affect the distribution if standard equal-width or equal-frequency binning methods were used. As

mentioned, our modeling approach can be used for any other type of sequential data that involves the duration of events, therefore the method used for categorizing the durations might need to be changed according to the data. 20 Duration Distribution for syscall_exit_connect First Second Window Eleventh Second Window 1200 1000 800 Frequency 600 400 200 0 10000 20000 30000 40000 50000 Duration (ns) 60000 70000 80000 90000 Figure 6: Distribution shift of “connect” syscall durations after 10 seconds of normal operation, demonstrating the need for duration category adjustments. 4.2.2. Duration Interval Adjustment Even under stable conditions, the distribution of event durations under goes minor variations. These slight shifts in event duration distributions, in f luenced by changes in the server’s load and operational context, could alter which events fall into each category. This may potentially lead to misrepre sentations or inaccuracies in the categorization. Therefore, these shifts need to be reflected in the intervals representing each of the duration categories to maintain the intended balance between the number of events assigned to each category and to ensure the model’s accuracy and effectiveness. Periodic up dates to the duration categories are thus necessary to adapt to these changes. Figure 6 illustrates how the distribution of connect durations evolves after 10 seconds, motivating the need for periodic adjustment. Considering these updates, LMAT blends new data with existing infor mation. For each event type, LMAT first checks if

there's existing data on event durations. If so, it calculates a weighted average of the event durations, combining the newly observed durations with the previously recorded ones. This calculation gives more weight to durations with a higher occurrence count, ensuring that the most common patterns have a greater influence on 21 the updated intervals. For new event types that weren't previously recorded, LMAT directly adopts the newly observed durations. Through this process, it dynamically adjusts the duration categories, integrating the latest obser vations while maintaining continuity with historical data. This lightweight, self-tuning procedure acts as a guardrail against distribution drift in execu tion times, ensuring that the event duration categories remain accurate and reflective of the server's current state, capturing both subtle and significant shifts while incurring minimal overhead. 4.3. Trace event selection Training models on all available syscall events is both inefficient and un necessary, as many system calls provide little insight into the system's inter nal state. Reducing the number of events used for training helps decrease model size and inference time. It also lowers the runtime overhead of the tracer during normal workloads, since only a relevant subset of events needs to be collected. To facilitate this minimization, we consider two separate sets of events in the LMAT design: one collected for monitoring purposes and the other for recording on disk for further analysis. As mentioned, during normal system operation,

events are not recorded; instead, they are fed into the model to detect any possible changes in behavior. Therefore, having a reduced set of events, specifically selected for change detection purposes, has a minimal impact on the overall process. However, selecting events for the second phase of analysis, conducted by a human operator, is not as straightforward, neces sitating the provision of as much information as possible about any potential issues. Accordingly, when abnormal behavior of the system is detected, a second set of events is considered for recording during the phase change, ensuring the inclusiveness of the trace To evaluate this strategy of event reduction, we considered various sub sets of events. These include 'select-top40', which consists of the 40 most frequently occurring events. To explore the impact of disregarding highly frequent but potentially less informative events, we also created subsets 'discard-top10' and 'discard-top20', which exclude the top 10 and top 20 most frequent events from the 'select-top40' list, respectively. Each of these subsets was selected with the goal of achieving a balance between providing thorough insight into the system's functioning and enhancing the model's efficiency, enabling it to respond to variations in system behavior in a timely 22 Table 3: The definition and size of each subset of system calls, designed to assess the feasibility of system monitoring with a reduced set of events. Subset Count Description select-top40 40 discard-top10 30 discard-top20 20 Top 40 system-calls with the

highest frequency are selected Top 10 events with the highest frequency are disre garded along with others removed in select-top40 Top 20 events with the highest frequency are disre garded along with others removed in select-top40 manner. A summary of these combinations and their total number of events is listed in Table 3.

This section explains how they turn raw kernel timestamps into duration labels, how they group those durations into categories, and why they keep updating those categories over time. The big idea is simple: measure how long each operation takes, convert those times into ordered groups like fast/medium/slow, and keep those groups updated because the system's timing naturally drifts over time.[49] [50]

Duration labeling

The duration of an operation is measured from its Entry event to its matching Exit event. The paper says these two events are matched using PID and TID, then the duration is computed as the difference between their timestamps, and that duration is attached to the Exit event while the Entry event gets a "none" duration tag.[50] [49]

In plain words:

- Entry = operation starts.[49]
- Exit = operation ends.[49]
- Duration = end time minus start time.[49]
- Tag duration on Exit, not on Entry, because the full time is only known at the end.[49]

Duration categories

They do not keep raw durations as exact numbers for training. Instead, they sort durations for each event type and divide them into k ordered bins using a custom decreasing-percentile strategy, so most samples fall into the fast category and fewer samples fall into slower categories.[49]

This is done because kernel durations are usually skewed: many events are very fast, and only a small number are slow. The paper wants the categories to preserve that real pattern instead of forcing artificial equal-sized or equal-width bins.[49]

The binning formula

The paper defines the total weight as $\text{Total} = \frac{k(k+1)}{2}$, which is the sum of the numbers from 1 to k. Then the fraction for bin i is $f(i) = \frac{k-i+1}{\text{Total}}$, which makes earlier bins larger and later bins smaller.[49]

Very simple example with $k = 3$:

- Total $= \frac{3(4)}{2} = 6$.[49]
- Bin 1 fraction $= \frac{3}{6} = 0.5$.[49]
- Bin 2 fraction $= \frac{2}{6} \approx 0.33$.[49]
- Bin 3 fraction $= \frac{1}{6} \approx 0.17$.[49]

So if you had 600 durations:

- Fast bin gets about 300.[49]
- Medium bin gets about 200.[49]
- Slow bin gets about 100.[49]

That matches the paper's goal: many fast events, fewer medium ones, and even fewer slow ones.[49]

Why update bins

Figure 6 shows that even during normal operation, the duration distribution can shift over time. Because of that, the original category boundaries may stop matching the system's current behavior, which can hurt accuracy and produce misleading labels.[50] [49]

So LMAT periodically adjusts the duration intervals:

- If an event type already has historical duration data, it blends old and new observations using a weighted average.[49]
- More frequent durations influence the update more strongly.[49]
- If the event type is new, LMAT just uses the newly observed durations.[49]

This is their way of handling distribution drift in execution times without expensive retraining. [50] [49]

Event selection

The last part explains that they do not want to train on every syscall event because that would be inefficient and may include many events that are not very informative. Reducing the event set can lower model size, reduce inference time, and reduce tracing overhead during monitoring.[49]

They evaluate three subsets:

- select-top40: the 40 most frequent system calls.[49]
- discard-top10: start from top 40, then remove the 10 most frequent ones, leaving 30 events. [49]
- discard-top20: start from top 40, then remove the 20 most frequent ones, leaving 20 events. [49]

The reason for discarding some very frequent events is that highly common events may dominate the data but add little useful information for detecting internal state changes.[49]

The easiest way to remember this whole section is:

- First, compute duration from Entry to Exit.[49]
- Second, convert durations into ordered bins, with more space for fast events.[49]
- Third, update those bins over time because timing distributions drift.[50] [49]
- Fourth, train only on a useful subset of events to save cost and improve efficiency.[49]

Would you like me to next explain the binning formula with a tiny worked example using actual duration numbers like 1000, 1200, 1400, 3000, 5000?

⁂

5. Evaluation In our experiment, we implement the proposed LMAT using LSTM, a recurrent neural network, and BERT [35], a Transformer-based architecture, both of which are commonly employed in tasks related to language model ing. These networks process a sequence of events representing a complete request, except some extreme cases which are truncated to 2048 events. The processing of complete requests is made possible by training the Transformer model with mixed precision [36] and gradient checkpointing [37], as these techniques increase memory efficiency, allowing for more effective handling of large sequences [12]. The implementation was carried out in Python using the Pytorch frame work [38]. We set the hyper-parameter λ at 0.5. The selection of other hyper-parameters is oriented towards optimizing the change detection task, which is the ultimate objective. To minimize the impact of random varia tion, each test was performed three times with distinct seeds. The training process takes place on a server provided by the Digital Research Alliance of Canada 4, equipped with two Intel Gold 6148 Skylake processors, four Nvidia V100SXM2 16G GPUs, and 64GB of RAM. 4https://alliancecan.ca 23 Table 4: Performance results of event and duration models trained separately in the multi task setup. LSTM-ST Event LSTM- MT Transformer- ST Dataset Transformer- MT Duration Event Duration Event Duration Duration CE Acc CE Acc CE Acc CE Acc CE Acc CE Acc CE Acc CE Acc CE Acc Event Train 0.71

76.2 0.04 91.6 0.83 76.6 0.06 86.0 0.85 72.9 0.03 91.6 0.95 72.4 0.08 81.6 Test ID 0.71 76.0 0.03 92.7 0.80 76.6 0.05 87.6 0.85 72.7 0.03 92.7 0.95 72.2 0.07 82.8 Test Connection 1.07 66.0 0.08 87.3 1.53 58.2 0.10 80.4 1.60 55.1 0.08 85.1 1.78 54.1 0.13 71.8 Test CPU 1.33 59.8 0.14 82.7 1.70 56.3 0.14 76.4 1.78 51.4 0.12 80.1 1.89 53.2 0.18 64.5 Test IO Test OPCache Test Socket Test Sysbench 2.33 40.2 0.22 78.3 2.75 38.4 0.21 71.0 3.46 27.0 0.13 76.7 3.91 24.4 0.28 51.1 1.26 61.6 0.08 87.2 1.49 60.3 0.09 81.3 1.45 59.2 0.07 87.2 1.56 59.2 0.10 76.8 1.48 55.0 0.12 82.8 1.83 51.9 0.15 74.3 1.91 47.7 0.10 77.9 2.03 49.1 0.19 61.1 0.90 71.6 0.10 85.4 1.22 67.1 0.10 81.8 1.12 67.1 0.08 84.4 1.25 66.4 0.10 76.3 Test Bandwidth 0.89 79.0 0.07 88.9 1.28 71.8 0.11 85.5 1.08 73.9 0.07 86.6 1.30 72.3 0.12 77.1 5.1. Event Sequence and Duration Modeling Language modeling and duration prediction tasks form the foundation of the Adaptive Tracer module of LMAT. The trained models are assessed based on the cross-entropy loss and top-one accuracy next token prediction, given a sequence of events or durations. These metrics effectively evaluate the model's predictive accuracy and its capacity for generalization. Event sequence and duration models are trained and evaluated both in a single-task and multi-task setup to assess the impact of the combination of their modeling objectives. The training results of event and duration models are given in Table 4. Duration models, particularly, demonstrated a notable ability to learn patterns of event completion durations, achieving an average accuracy of 92.7% in the in the normal test trace (Test ID) with 7

duration categories. The ability of the duration models to effectively utilize the same features reinforces the potential of a multi-task learning setup. A consistent observation across both the event and duration prediction models is a decline in predictive performance when encountering out-of distribution datasets. This characteristic is crucial for detecting novel sys tem behaviors, as significant deviations from the model's predictions can signal unusual activities. The decrease in performance on the Sysbench and Bandwidth datasets was less pronounced for the event models, highlighting the limitations of relying solely on the event sequence model for capturing anomalies, particularly those related to event durations. In contrast, dura tion models exhibited increased sensitivity to these datasets, suggesting that they are more adept at identifying such deviations. 24 Despite single-task models excelling in next-token prediction accuracy compared to their multi-task counterparts, this superiority did not always translate into better outcomes for change detection or root cause analysis. Our subsequent analysis will demonstrate that multi-task training fosters more generalized models, thereby enhancing the detection of unknown be haviors. This improvement is particularly evident in the increased prediction loss for the event prediction component in multi-task models when evaluated on the Sysbench and Bandwidth datasets. For instance, in the LSTM model applied to the Bandwidth dataset, the prediction loss difference widened from (0.71, 0.89) to (0.8, 1.28),

highlighting the effectiveness of a unified approach for identifying abnormal behaviors.

This section explains how they tested LMAT, what metrics they used, and how to read Table 4. The main takeaway is that they trained both LSTM and Transformer models for event prediction and duration prediction, then checked how well those models predict normal traces versus out-of-distribution traces, because worse prediction on unusual data is exactly what helps anomaly detection.[51]

What they evaluated

The paper says LMAT was implemented using two sequence models: LSTM and BERT-style Transformer. These models process each request as a sequence of events, and very long requests were truncated to 2048 events so training stayed manageable.[51]

They also used mixed precision and gradient checkpointing so the Transformer could handle longer sequences more efficiently in memory. In simple terms, these are engineering tricks that let a large model fit and train on long traces without running out of GPU memory too quickly.[51]

The metrics

Table 4 reports two main metrics for both the event model and the duration model:

- CE, which means cross-entropy loss.[51]
- Acc, which means top-1 accuracy.[51]

You can read them like this:

- Lower CE is better, because it means the model assigned higher probability to the true answer.[51]
- Higher accuracy is better, because it means the model predicted the correct next event or duration category more often.[51]

A very important idea in anomaly detection is that performance should drop on unusual datasets. If CE gets higher or accuracy gets lower on anomalous traces, that is actually useful because it means the model notices that the new behavior does not look normal.[51]

Table 4 meaning

The table compares four setups:

- LSTM-ST, which means LSTM single-task.[51]
- LSTM-MT, which means LSTM multi-task.[51]
- Transformer-ST, which means Transformer single-task.[51]
- Transformer-MT, which means Transformer multi-task.[51]

And for each setup, the paper reports results for:

- Event prediction.[51]
- Duration prediction.[51]

So each row tells you how one architecture-task setting performs on one dataset such as Train, Test ID, Test CPU, or Test Bandwidth.[51]

How to read one row

Take the normal test set, Test ID. For LSTM-ST, the event model has CE 0.71 and accuracy 76.0, while the duration model has CE 0.03 and accuracy 92.7, which means the duration prediction is much stronger than event prediction on that normal data.[51]

That is why the paper says the duration models showed strong learning ability. In their normal test trace, duration prediction reached 92.7% average accuracy with 7 duration categories, which is very high for a multi-class prediction task.[51]

What changes on anomalies

Across the out-of-distribution datasets, predictive performance drops. The paper says this decline is expected and useful, because abnormal behavior causes the model to be more surprised, which raises loss and lowers accuracy.[51]

For example, on harder anomaly datasets like IO, CPU, Socket, or Connection, CE rises and accuracy falls compared with Test ID. That means the model's expectations learned from normal traces do not match these abnormal traces as well.[51]

The paper also highlights that event-only models were less affected on Sysbench and Bandwidth, which suggests event order alone may miss some anomalies. Duration models were more sensitive there, so timing information helped detect changes that event sequences did not capture well.[51]

Single-task vs multi-task

The paper says single-task models often had better next-token prediction accuracy than multi-task models. But higher prediction accuracy on the base task did not always mean better anomaly detection or better root cause analysis.[51]

Their argument is that multi-task learning gives more generalized models. In anomaly detection, a model that becomes more surprised on unknown behaviors can sometimes be more useful than one that is only slightly better at standard prediction on known normal traces.[51]

The example they give is LSTM on the Bandwidth dataset:

- In single-task event prediction, the loss changed from 0.71 on Test ID to 0.89 on Test Bandwidth.[51]
- In multi-task event prediction, the loss changed from 0.80 on Test ID to 1.28 on Test Bandwidth.[51]

That larger jump means the multi-task model reacted more strongly to the abnormal bandwidth behavior, which is helpful for change detection.[51]

The simplest way to remember this section is:

- Accuracy tells how often the model guessed right.[51]
- Cross-entropy tells how confident and correct the model was.[51]
- Good anomaly detectors often look worse on abnormal data, because abnormal data should surprise them.[51]
- Multi-task models may sacrifice some plain prediction accuracy but become better at noticing unusual behavior.[51]

Would you like me to turn Table 4 into an easy comparison table with only the most important rows and what they mean?

⁂

5.2. Change detection In the change detection task, the LMAT model evaluates each request to classify it as either normal or novel. This classification is based on measuring the cross-entropy loss of each request and comparing it to a predetermined threshold. This threshold is selected by considering the losses of both normal and noisy requests available in the validation sets, with the goal of maximiz ing the F1-score for this binary classification. The F1-score is the harmonic mean of precision and recall metrics, ranging from 0 to 1, where 1 indicates perfect precision and recall, and 0 signifies that either precision or recall is zero. The model's performance is also assessed using the Area Under the ROCCurve (AUC), which provides a comprehensive measure of effectiveness across all potential classification thresholds, offering a broad perspective on the model's predictive capabilities. To understand how duration categorization impacts the model's ability to detect changes, we explore various numbers of categories, assessing the model's performance across a range of 3 to 9 categories. Based on the re sults detailed in Table 5, employing 7 categories provided the most effective balance, establishing our standard for subsequent experiments. Expanding beyond this number leads to a decline in model performance, likely due to the introduction of excessive complexity that does not proportionately enhance predictive capability, ultimately reducing the model's efficiency. Tables 6 and 7 present a comparative

analysis of change detection out comes between single-task and multi-task configurations for LSTM and BERT models, respectively. The integration of duration modeling is shown to significantly improve change detection capabilities. Importantly, the du ration model not only captures all changes recognized by the event model 25 Table 5: Change detection results for different numbers of duration categories. 3 Category 5 Category Test Set 7 Category F1 AUC F1 AUC F1 AUC 9 Category AUC Connection 94.5 ± 0.8 98.7 ± 0.4 97.9 ± 2.9 99.5 ± 0.7 98.0 ± 1.8 99.7 ± 0.3 99.1 ± 0.2 99.9 ± 0.0 CPU F1 99.7 ± 0.2 99.9 ± 0.1 99.7 ± 0.2 100.0 ± 0.1 99.5 ± 0.4 99.9 ± 0.1 99.0 ± 0.3 99.9 ± 0.1 IO OPCache Socket Sysbench 99.9 ± 0.0 100.0 ± 0.0 99.9 ± 0.0 100.0 ± 0.1 99.9 ± 0.0 100.0 ± 0.1 99.4 ± 0.3 100.0 ± 0.1 93.6 ± 1.3 98.3 ± 0.7 97.6 ± 1.8 99.6 ± 0.3 97.9 ± 1.4 99.7 ± 0.2 96.2 ± 2.2 99.4 ± 0.5 98.3 ± 0.8 99.3 ± 0.5 98.8 ± 0.7 99.7 ± 0.2 98.4 ± 1.0 99.5 ± 0.4 97.7 ± 2.0 99.7 ± 0.3 97.8 ± 2.1 99.4 ± 0.7 97.7 ± 2.1 99.5 ± 0.6 98.5 ± 1.8 99.7 ± 0.4 99.1 ± 0.5 99.9 ± 0.1 Bandwidth 66.6 ± 0.0 46.0 ± 5.0 87.1 ± 3.9 94.1 ± 3.2 94.9 ± 3.0 98.6 ± 1.1 88.6 ± 9.3 99.0 ± 0.7 Average 92.9 ± 0.2 91.7 ± 0.7 97.0 ± 1.4 98.9 ± 0.6 98.2 ± 1.2 99.6 ± 0.3 97.0 ± 1.9 99.7 ± 0.2 Table 6: Change detection performance: LSTM single-task versus multi-task. Test Set Event Duration Multi-task Connection 98.7 ± 0.5 98.0 ± 1.8 99.6 ± 0.1 CPU 99.0 ± 0.4 IO OPCache 99.2 ± 0.2 99.5 ± 0.4 99.6 ± 0.2 99.9 ± 0.0 99.9 ± 0.1 98.8 ± 0.3 97.9 ± 1.4 98.6 ± 0.4 Socket Sysbench 98.6 ± 0.8 98.4 ± 1.0 99.1 ± 0.5 88.3 ± 2.9 98.5 ± 1.8 97.3 ± 0.8 Bandwidth 90.2 ± 6.5 94.9 ± 3.0 96.1 ± 0.7 Average 96.1 ± 1.6 98.2 ± 1.2

98.6 ± 0.1 but also uncovers further notable variations, especially within the Sysbench and Bandwidth datasets. This increased detection is attributed to the dis tinctively higher loss values that the duration model associates with these datasets. While the duration model proves more effective in several scenarios, there are instances where the event model excels, indicating the value of a combined approach. In fact, the choice between these models is context-dependent, and neither can be deemed universally superior. This highlights the need for a holistic modeling approach to capture the full spectrum of system behavior changes. Multi-task models address this need effectively, achieving com prehensive change detection with minimal performance trade-offs, given the marginal increase in parameters. Furthermore, these models exhibit greater stability in their outcomes compared to their single-task counterparts, as reflected by reduced variability in their performance metrics. When comparing the LSTM and Transformer models, it is notable that the LSTM, despite its relative simplicity, competes closely with, and in some cases surpasses, the more resource-intensive Transformer model. This com parable performance suggests that the LSTM's strength in capturing local 26 Table 7: Change detection performance: BERT single-task versus multi-task. Test Set Event Duration Multi-task Connection 99.2 ± 0.7 98.5 ± 0.8 99.7 ± 0.0 CPU 99.1 ± 0.7 IO OPCache 99.3 ± 0.9 98.6 ± 0.4 99.5 ± 0.4 98.8 ± 0.4 100 ± 0.0 98.2 ± 0.6 95.5 ± 3.2 98.0 ± 0.8 Socket

Sysbench 98.0 ± 0.7 99.3 ± 0.1 99.5 ± 0.3 87.4 ± 4.2 97.1 ± 1.9 91.7 ± 0.4 Bandwidth 63.7 ± 15.1 97.4 ± 2.2 94.0 ± 4.8 Average 92.1 ± 1.3 97.9 ± 0.4 97.5 ± 0.9 dependencies is particularly beneficial in this case, somewhat mitigating the Transformer's advantage in broader contextual understanding. However, it is important to consider scenarios involving system traces with patterns ex tending over larger spans, where the Transformer model's capacity for wider context might prove more advantageous. We further validate our LMAT methodology using the dataset from [12] to benchmark against their event sequence-based novelty detection method, as well as to extend duration modeling to an additional dataset. A criti cal distinction is that [12] optimized their model performance by applying a separate threshold for each out-of-distribution set, which diverges from our methodology. For a more realistic scenario approximation, where multiple thresholds for change detection aren't practical due to the unpredictable na ture of changes, we employed a single threshold strategy that maximizes the F1-score across all out-of-distribution validation sets for their models, align ing with our approach. This adjustment in methodology leads to different re sults from those reported by [12], as detailed in Table 8. Our findings demon strate that the multi-task LSTM and BERT models surpass the performance of the event sequence models by [12] in most sets. This improvement pre dominantly stems from the explicit integration of duration modeling. On the other hand, the method by [12], incorporating

elapsed time between events into their event representation to capture duration variations, falls short in accurately reflecting the comprehensive dynamics of duration changes.

This section explains how they turn model loss into a yes/no decision: normal or novel. The model computes a loss for each request, compares it with a threshold chosen on the validation data, and if the loss is high enough, the request is classified as abnormal.[52] [53]

Core idea

The paper uses cross-entropy loss as the abnormality signal. A low loss means the trace looks like normal behavior the model has learned, while a high loss means the model is surprised, so the trace may be novel or anomalous.[53] [52]

The threshold is selected using validation data from both normal and noisy requests. They choose the threshold that gives the best F1-score for the binary task of separating normal from abnormal traces.[52] [53]

F1 and AUC

The paper evaluates change detection with F1-score and AUC. F1 is the harmonic mean of precision and recall, so it is useful when you want one number that balances "how many detected anomalies were truly anomalies" and "how many true anomalies were actually found."[53] [52]

AUC means Area Under the ROC Curve. In simple language, it measures how well the model separates normal and abnormal samples across many possible thresholds, not just one fixed threshold.[52] [53]

A simple way to remember them:

- F1 = quality at one chosen threshold.[52]
- AUC = overall separation ability across all thresholds.[52]

Table 5

Table 5 studies how many duration categories should be used. The paper tests 3, 5, 7, and 9 categories and reports that 7 categories gave the best balance overall, with average F1 of 98.2 and average AUC of 99.6.[52]

The reason is that too few categories may be too coarse to capture timing changes, while too many categories may add unnecessary complexity and hurt performance. In their results, moving from 7 to 9 categories slightly reduced average F1 even though AUC stayed very high.[52]

Tables 6 and 7

Tables 6 and 7 compare event-only, duration-only, and multi-task models for LSTM and BERT. The main pattern is that duration modeling often improves change detection, especially on datasets like Sysbench and Bandwidth where timing changes matter a lot.[53] [52]

For LSTM in Table 6:

- Average Event = 96.1.[52]
- Average Duration = 98.2.[52]
- Average Multi-task = 98.6.[52]

For BERT in Table 7:

- Average Event = 92.1.[53]
- Average Duration = 97.9.[53]
- Average Multi-task = 97.5.[53]

So duration is clearly stronger than event-only on average in both model families, and multi-task is usually the most balanced overall approach.[53] [52]

Why duration helps

The paper says the duration model catches not only the changes found by the event model, but also extra variations, especially in Sysbench and Bandwidth. That happens because some anomalies do not change the event order very much, but they do change how long operations take.[53] [52]

You can see this clearly in the BERT results:

- On Bandwidth, Event = 63.7, Duration = 97.4, Multi-task = 94.0.[53]
- On Sysbench, Event = 87.4, Duration = 97.1, Multi-task = 91.7.[53]

That is a strong sign that event order alone can miss important timing-related anomalies.[53]

LSTM vs Transformer

The paper notes that LSTM performs competitively with, and sometimes better than, the Transformer despite being simpler. Their interpretation is that local sequential dependencies are very important in this setting, so LSTM can be very effective even without the heavier architecture of a Transformer.[52] [53]

For example, the average multi-task score is 98.6 for LSTM and 97.5 for BERT in the shown tables. That suggests the simpler model was not only cheaper but also slightly better in this experiment.[52] [53]

The simplest study note for this section is:

- Use loss as an anomaly score.[52]
- Choose threshold on validation data.[52]

- Measure performance with F1 and AUC.[52]
- Duration information is very important for detecting timing anomalies.[53] [52]
- Multi-task usually gives the most complete view of abnormal behavior.[53] [52]

Would you like me to explain F1, precision, recall, threshold, ROC, and AUC with one tiny confusion-matrix example next?

⁂

5.3. Root cause analysis The performance of root cause analysis, framed as a classification task, is quantified through accuracy metrics. As outlined in Section 3, to repre sent an abnormal sequence within the scope of a multi-task model, options 27 Table 8: Performance comparison of LMAT models versus Fournier et al. [12] (F LSTM and F BERT) using a unified threshold strategy. Test Set LSTM BERT F LSTM FBERT Connection 97.3 ± 1.1 94.4 ± 2.2 62.8 ± 6.9 94.1 ± 3.3 CPU 98.8 ± 0.5 92.5 ± 1.5 94.1 ± 0.9 81.6 ± 8.4 IO OPCache Socket SSL Average 99.9 ± 0.1 99.9 ± 0.0 95.0 ± 0.5 98.2 ± 0.6 96.5 ± 0.9 93.7 ± 1.5 94.8 ± 0.5 96.6 ± 0.6 99.6 ± 0.1 99.0 ± 0.2 93.9 ± 0.8 98.0 ± 0.6 99.2 ± 0.4 98.6 ± 0.6 85.3 ± 2.3 97.8 ± 1.0 98.6 ± 0.5 96.4 ± 0.9 87.7 ± 2.5 94.4 ± 1.8 Table 9: Evaluating root cause classification strategies within the multi-task model frame work. Data Source Average Accuracy (%) Event Vec 97.6 ± 0.2 Duration Vec Mean Vec 99.0 ± 0.8 97.6 ± 0.4 include utilizing mispredicted events, mispredicted durations, or a combi nation of their resulting vectors. To determine the most effective method for root cause classification in further experiments, we initially analyze the root cause analysis module in isolation, focusing solely on the accuracy of root cause classification after changes in the system have been detected. Ac cording to Table 9, each approach for generating sequence representations effectively facilitates the correlation of newly observed abnormalities with established abnormal patterns. Given the slightly enhanced performance observed with

representations generated from mispredicted durations, this technique is adopted for conducting root cause analysis using the multi-task model framework. As root cause analysis operates on top of the change detection task, the overall performance of this module should be assessed by considering both tasks as a whole. In other words, detecting a change is a prerequisite for root cause identification; without detecting the change, no root cause can be identified for a false negative. Referring to Table 10, it is evident that both the event and duration modeling approaches are effective in accurately identifying changes and their root causes. This observation aligns with the change detection task, where the duration model sometimes outperforms the event model and vice versa. However, the duration model generally exhibits superior performance, achieving an average accuracy of 96.9% with 28 Table 10: Integrated performance of root cause analysis and change detection. Test Set Event Duration Multi-task Connection 97.9 ± 0.7 96.9 ± 1.9 98.2 ± 1.9 CPU 97.4 ± 0.3 IO 97.7 ± 0.1 98.7 ± 0.6 100 ± 0.0 99.9 ± 0.0 99.9 ± 0.1 OPCache Socket Sysbench 98.5 ± 0.4 97.2 ± 0.5 96.1 ± 2.5 92.2 ± 0.1 96.6 ± 1.2 98.1 ± 1.3 79.9 ± 4.6 97.0 ± 1.6 95.7 ± 1.1 Bandwidth 81.4 ± 12.2 92.8 ± 5.8 96.7 ± 1.1 Average 92.5 ± 2.5 96.9 ± 1.5 97.7 ± 1.0 less variability in results. On average, the multi-task model outperforms each of the single-task models by capitalizing on their individual advantages, resulting in more consistent and reliable performance across various noise scenarios.

This section evaluates how well the system identifies the **type** of anomaly after an abnormal trace has been found. The important idea is that root cause analysis depends on change detection first, so the real performance must consider both steps together: first detect that something is wrong, then classify what kind of problem it is.[54] [55]

What is measured

Here root cause analysis is treated as a classification task, so the metric is accuracy. Accuracy here means the percentage of abnormal traces for which the system assigns the correct anomaly label.[55] [54]

The paper studies different ways to represent an abnormal sequence:

- Using mispredicted events.[54]
- Using mispredicted durations.[54]
- Using a combined vector.[54]

These representations are then used to match the abnormal trace to known anomaly patterns.[54]

Table 9

Table 9 compares the root-cause representation strategies inside the multi-task framework. The duration-based vector gives the best average root cause classification accuracy at 99.0%, compared with 97.6% for the event vector and 97.6% for the mean vector.[55] [54]

That tells us something important: timing mistakes were slightly more informative than event-sequence mistakes for identifying which anomaly type occurred. In other words, duration deviations carried the clearest signature for root cause classification in these experiments.[55] [54]

Why detection matters first

The paper explains that root cause analysis sits on top of change detection. That means if the system fails to detect an anomaly at all, it cannot possibly identify the root cause for that missed case.[54]

So there are really two stages:

1. Detect the abnormal trace.[54]
2. Classify the root cause of that detected abnormal trace.[54]

Because of this dependency, the paper says the final evaluation should combine both tasks rather than looking only at root-cause classification in isolation.[54]

Table 10

Table 10 reports this integrated performance, meaning detection plus root-cause identification together. The average scores are 92.5 for the event model, 96.9 for the duration model, and 97.7 for the multi-task model.[55] [54]

So the ranking is:

- Best overall: multi-task model, 97.7 average.[54]
- Second: duration model, 96.9 average.[54]
- Third: event model, 92.5 average.[54]

This supports the paper's main message that combining event and duration information produces the most reliable system overall.[55] [54]

How to read dataset differences

The best model can vary by dataset, which the paper also points out. For example, the event model is strongest on IO with 100.0, while the duration model is stronger on Sysbench with 97.0 versus 79.9 for the event model, and the multi-task model is strongest on Bandwidth with 96.7.[54]

This means no single signal is always best:

- Some anomalies are better revealed by event-order mistakes.[54]
- Some are better revealed by timing mistakes.[54]
- The multi-task approach works well because it can benefit from both.[55] [54]

Table 8

Table 8 compares LMAT against earlier models from Fournier et al. using one unified threshold strategy. In that comparison, LMAT's LSTM has the best average score at 98.6, which is higher than LMAT's BERT at 96.4, F_BERT at 94.4, and F_LSTM at 87.7.[55] [54]

This is useful because it shows the proposed method is competitive not only internally but also against prior work. The paper argues that the gain comes mainly from explicit duration modeling, rather than trying to capture time changes only indirectly through event representation.[55] [54]

The easiest study note for this section is:

- Table 9 asks: "Which error representation is best for root-cause classification?" Answer: duration vector.[54]
- Table 10 asks: "Which whole system is best when detection and root-cause analysis are both required?" Answer: multi-task.[54]
- Table 8 asks: "How does LMAT compare with prior work?" Answer: generally better, especially with LSTM.[55] [54]

Would you like me to make a one-page cheat sheet for all metrics you have seen so far in this paper, like CE, accuracy, precision, recall, F1, AUC, MAD, cosine similarity, and threshold?

⁂

5.4. Tracing The overall effectiveness of the Adaptive Tracer module of LMAT is evalu ated based on its accuracy for the selection of traces related to a new pattern and the volume of trace data it manages to reduce. Recorded trace data volume is used as a general measurement of the tracing overhead, especially on disk space usage. Evaluation is based on a set of test traces with vary ing levels and types of noise, giving us a comprehensive view of the model's performance under different conditions. These test traces are created by randomly distributing already collected noisy requests among the requests from traces of normal workload, to simulate changes in system behavior, and each of the noisy setups includes more than 6,000 abrupt changes. Model processes trace data per request and using request as a unit to create test scenarios helps with keeping the integrity of the collected trace and model's input. To determine when to initiate tracing, the Adaptive Tracer employs a window-based decision-making strategy that defers recording until the pro portion of requests identified as abnormal within a window exceeds a prede termined threshold. The window size is set to match the batch size (16), with the threshold established at 80% of the window size. This method ensures that only significant deviations prompt recording, enhancing the tracer's pre cision in capturing data shifts. Responsiveness to system changes is another crucial aspect of Adaptive Tracer that we look into. Based on the

test scenarios, we measure the time 29 interval from the onset of a major change burst to the end of the first win dow in which the tracer decides to start recording, excluding any processing latencies. This time allows for a comparison of the modeling approaches in terms of their effectiveness at capturing relevant details of a sudden issue within the system. Figure 7 illustrates a comparison between the event and multi-task mod els' performance in adaptive tracing across different noise levels within the Sysbench dataset. Despite the event model showing a 9% lower performance in change detection for this particular dataset, it achieves a significantly higher miss rate for relevant data and a more pronounced delay in detecting changes, with disparities of 30.3% and 14.2ms (56.8%) respectively. Con versely, the multi-task model maintains an average change detection delay of 25ms with only a 1.5% data loss. Independent of the base model, adaptive tracing leads to a substantial decrease in the amount of trace data stored in the system. When the trace contains no noisy requests, the volume of data is reduced by approximately 99%, as the model detects almost no significant deviations. In general, the event model achieves a 10.1% greater reduction in trace data; however, this comes at the expense of an increased loss of critical trace information. Delays in detecting changes have a direct impact on the volume of data missed during the adaptive tracing process. In Figure 8, the comparison of miss rates across various modeling approaches underscores

the critical need for precise change detection. While 3.2% trace related to novel behaviors are lost in all the test scenarios utilizing the multi-task model, the loss is more than 3 times higher when the Adaptive Tracer functions based on event sequence model. The multi-task model achieves such precision alongside a 70.6% reduction in recorded trace volume across all out-of-distribution datasets.

This section evaluates the adaptive tracing part of LMAT, which decides **when** to start recording traces instead of recording everything all the time. The core idea is to reduce tracing overhead while still capturing important abnormal behavior quickly and accurately.[56]

What they measure

The paper evaluates adaptive tracing using three main ideas:

- How much trace data is reduced.[56]
- How much important abnormal trace data is missed.[56]
- How long it takes to react after a major behavior change starts.[56]

In simple words:

- Trace reduction tells you how much storage overhead was saved.[56]
- Miss rate tells you how much important anomaly-related trace data was lost.[56]
- Delay tells you how fast the tracer starts recording after the anomaly begins.[56]

Window decision

The tracer does not start recording after a single suspicious request. Instead, it uses a window-based rule: look at a batch of 16 requests, and start recording only if at least 80% of that window is classified as abnormal.[56]

So the logic is:

- Window size = 16 requests.[56]
- Threshold = 80% of the window.[56]
- 80% of 16 is 12.8, so in practice the decision means about 13 abnormal requests in the window are needed before recording starts.[56]

This helps avoid reacting to tiny random fluctuations. The tracer records only when there is a strong sign that system behavior has really changed.[56]

Delay meaning

The delay metric measures the time from the start of a major anomaly burst until the end of the first decision window that triggers recording. The paper explicitly says this excludes processing latency, so it is focusing on decision responsiveness rather than implementation overhead.[56]

Why this matters:

- Shorter delay means the system starts saving relevant traces sooner.[56]
- Longer delay means more abnormal requests may happen before tracing begins.[56]
- That usually increases missed anomaly data.[56]

Figure 7

Figure 7 compares the event-only model and the multi-task model on Sysbench noise. The paper says the event model has a much higher miss rate and larger delay, while the multi-task model keeps both values lower.[56]

The reported differences are:

- The event model has a 30.3% higher miss rate.[56]
- The event model has 14.2 ms more delay, which the paper describes as 56.8% higher.[56]
- The multi-task model maintains about 25 ms average detection delay with only 1.5% data loss.[56]

That means the multi-task model reacts faster and misses less important abnormal trace data.[56]

Trade-off

Both models reduce stored trace volume a lot, especially when there is no noise. The paper says when the trace contains no noisy requests, recorded trace volume drops by about 99% because the model sees almost no reason to start recording.[56]

But there is a trade-off:

- The event model reduces trace data more aggressively.[56]
- Specifically, it achieves 10.1% greater reduction in trace volume.[56]
- However, that extra reduction comes at the cost of missing more important abnormal data.[56]

So saving more storage is not automatically better if you lose the traces you actually needed for diagnosis.[56]

Main conclusion

The paper argues that precise change detection is critical for adaptive tracing. Their multi-task model loses only 3.2% of anomaly-related trace data across test scenarios, while the event-based version loses more than three times as much, and still achieves a 70.6% reduction in recorded trace volume across all out-of-distribution datasets.[56]

The easiest way to remember this section is:

- Adaptive tracing tries to record less data.[56]
- But it must still catch abnormal periods quickly.[56]
- Miss rate and delay are the key practical metrics.[56]
- The multi-task model gives a better balance: less missed anomaly data with strong trace reduction.[56]

Would you like me to make a final cheat sheet for this whole paper with every metric explained in one line: CE, accuracy, F1, AUC, MAD, cosine similarity, miss rate, delay, and trace reduction?

⁂

5.5. Trace event reduction To assess the feasibility of having a secondary set of events for constant monitoring of the system with reduced overhead, we trained the model using the subsets of system calls mentioned in Table 3. The aggregated results from training the multi-task model on these subsets are presented in Table 11. As we can see, most frequently occurring events are not always the richest source of information for monitoring system behavior and tracking its trends. For instance, excluding the top-10 most repeated event types, only results in a 7% decline in change detection accuracy, despite a substantial 76% reduction in the volume of input data. This accuracy could potentially be preserved or even enhanced by selectively targeting specific events tailored to particular monitoring needs. These findings confirm that LMAT maintains strong performance even when operating on substantially reduced input sets, supporting the viability of a two-tier monitoring framework with a lightweight default mode and a more comprehensive tracing mode triggered on demand

Table 11: Evaluation of multi-task model accuracy on reduced system call subsets. Subset Change (F1) Root cause (Acc) Volume (%) select-top40 discard-top10 discard-top20 96.5 ± 1.2 91.6 ± 0.6 84.5 ± 1.2 96.8 ± 0.6 91.4 ± 0.4 74.4 ± 4.8 97 24 4 Normal 98.6 ± 0.5 97.5 ± 0.8 100

This section asks a practical question: can the system monitor behavior using only a smaller subset of syscall events, so tracing is cheaper, while still keeping good detection performance. The answer from Table 11 is yes to a point: LMAT still performs well with reduced event sets, but if you remove too much information, both change detection and root-cause accuracy start to drop clearly.[57]

Main idea

The paper is testing a two-tier monitoring design. The idea is to use a lightweight set of events all the time for cheap monitoring, and then switch to richer tracing only when abnormal behavior is detected.[57]

This matters because collecting every possible syscall all the time can be expensive. If a smaller event subset still preserves most of the useful signal, the monitoring system becomes much more practical.[57]

Table 11 meaning

Table 11 compares several subsets of system calls using three columns:

- Change (F1), which measures anomaly detection quality.[57]
- Root cause (Acc), which measures root-cause classification accuracy.[57]
- Volume (%), which shows how much input data remains compared with the full normal setting.[57]

The rows are:

- select-top40: use the 40 most frequent system calls.[57]
- discard-top10: start from those 40 and remove the 10 most frequent ones.[57]
- discard-top20: remove the 20 most frequent ones.[57]
- Normal: full event set baseline.[57]

What the numbers say

The full normal setting gives Change F1 of 98.6 and Root cause accuracy of 97.5 with 100% volume. The select-top40 setting stays close, with 96.5 F1 and 96.8 root-cause accuracy, while using 97% volume.[57]

The really interesting row is discard-top10:

- Change F1 = 91.6.[57]
- Root cause accuracy = 91.4.[57]
- Volume = 24%.[57]

That means the model keeps fairly strong performance while using only about one quarter of the original input volume. This is why the paper says excluding the top-10 most repeated events causes only about a 7% drop in change detection accuracy while reducing input volume by 76%.[57]

Why frequent events are not always best

The paper says the most common events are not always the most informative for monitoring. Very frequent events can dominate the trace simply because they happen often, but they may not tell you much about important behavioral changes.[57]

So removing some top-frequency events can sometimes improve efficiency a lot with only a moderate loss in accuracy. The monitoring signal may still be preserved because less frequent but more meaningful events remain.[57]

Where it breaks down

The discard-top20 setting shows the limit of reduction. Its Change F1 drops to 84.5 and Root cause accuracy drops to 74.4, while the volume is only 4%.[57]

So this setting is very lightweight, but it throws away too much useful information. In simple terms, the system becomes cheap, but now it is much less reliable for both detecting anomalies and identifying their causes.[57]

Practical meaning

The results support the paper's two-tier idea:

- Use a reduced subset for continuous low-overhead monitoring.[57]
- When a change is detected, switch to richer tracing for detailed analysis.[57]

The best compromise in Table 11 looks like discard-top10, because it cuts the input volume dramatically while keeping reasonably strong detection and root-cause performance.[57]

A very short way to remember this section is:

- Less data can still work.[57]
- Some common events are not very informative.[57]
- Moderate reduction is good.[57]
- Extreme reduction hurts accuracy too much.[57]

Would you like me to now make the full cheat sheet of all metrics and notations from this paper in simple words?

⁂

5.6. Overhead analysis The latency involved in executing the model's steps for adaptive tracing is a critical factor in assessing its feasibility in real-world scenarios. The laten cies for the various modeling approaches used in this study are documented in Table 12, measured using a V100 GPU. The sequence prediction phase is the most time consuming step, with a slight increase in latency observed in the multi-task approach due to the additional operations related to duration prediction task. Notably, reducing the number of monitored event types by considering the discard-top10 subset, decreases the model's inference time by 41%, which is the result of the substantial reduction in the length of input sequences and the number of potential events for prediction. With batching of 16 requests, the reduced LSTM configuration processes a batch in ~19.3ms, i.e., ~830 requests/s effective throughput on this (non tuned) setup. In our client workload (~1,000 req/s), the model is unable 32 to keep up with the stream of data. In practice, there are options to ad dress this and to improve the efficiency and throughput: (i) lighter hardware (e.g., mid-range GPU or CPU) when the application's latency budget allows; (ii) routine systems optimizations (reduced precision and larger batches) to improve throughput (usually yielding significant speedups in practice) and (iii) sampling the monitor stream (e.g., analyze p% of requests) when full coverage is unnecessary, because LMAT's detection accuracy is high even at modest sampling rates. 5.6.1.

Practical deployment target We emphasize that LMAT does not require high-end GPU clusters. We used V100s because they were available on our shared compute infrastruc ture; however, the LSTM variant, which in our experiments matches the Transformer variant in detection accuracy, has ∼1M parameters (∼5MB on disk) and occupied well under 1GB of device memory for inference. This footprint enables deployment on modest accelerators (e.g., entry-level data center/consumer GPUs) and, where latency budgets permit, on CPUs only. The model storage is a constant overhead; LMAT's savings accrue on the ongoing tracing stream. 5.6.2. Cost–benefit framing To compare compute overhead with storage savings, let: SavedStorage/s = R ×S ×ρ where R is requests/s, S is average raw trace size per request, and ρ is LMAT's fractional reduction (e.g., 0.706 for 70.6% reduction). Monthly stor age saved is Δstorage/month = SavedStorage/s × 2.592 × 106 (s/month). Let Cstorage denote the per-GB-month storage price and Ccompute the monthly cost of the inference host. The break-even condition is Ccompute ≤ Δstorage/month × Cstorage. Practitioners can instantiate S and Cstorage from their environment and pick the appropriate inference tier: 33 • CPU-only tier. For moderate R (hundreds req/s), the 1M-parameter LSTM typically fits per-core budgets; cost is amortized across existing servers. • Low-cost GPU tier. For higher R (thousands req/s), a small GPU (well below V100 class) provides ample headroom; device power draw is modest compared to top-end

accelerators. LMAT curbs unbounded trace growth. Even when storage is “cheap”, re tention, replication, backup, indexing, and downstream analytics costs scale with volume, and LMAT directly reduces all of them. Example (our setup). For 2,000,000 requests, the trace volume totals 35GB. Thus S = 35 2 ×106 GB/req ≈ 1.75×10–5 GB/req (~ 17.5 KB/req). With R ≈1000 req/s and **ρ** = 0.706: SavedStorage/s ≈ 1000 ×1.75 ×10–5 ×0.706 ≈ 1.236×10–2 GB/s. Over a month (2.592 ×106 s): **Δ**storage/month ≈ 0.01236 × 2.592 × 106 ≈ 32,024 GB (~ 31.3 TB). Using typical per-GB-per-month storage pricing5 : ⊠⊠⊠ Cstorage ∈ ⊠⊠ $0.012 Cloud object storage (Infrequent Access tier) $0.020 On-premises SAN/HDD estimate $0.023 Cloud object storage (Standard access tier) the monthly storage savings is $384–$737 (32,024 GB × Cstorage). Our recommended deployment is the ~1M-parameter LSTM (5 MB on disk), which matched the Transformer’s accuracy/delay in our experiments and ran well under 1 GB memory. Two realistic tiers: • CPU-only: for hundreds of req/s, incremental cost is typically negli gible on existing hosts. 5According to public cloud storage pricing (US, 2025) and on-premises shared storage cost models. 34 • Low-cost GPU (e.g., T4-class): Capital Expenditure ~ $1500 (as listed at the time of writing6) amortized over 36 months ⇒ ~ $42/mo; power ~ 70W ⇒~50kWh/mo≈$6 at $0.12/kWh; total ~ $48/mo.7 Even at conservative storage prices and without replication, monthly sav ings ($384–$737) exceed a low-cost GPU’s ~ $48/mo; with replication or downstream indexing,

the margin is substantially larger. Also, CPU-only de ployments have near-zero incremental compute cost. This shows that LMAT is economically viable in settings where raw trace volume, retention windows, or downstream analytics costs dominate.

This section checks whether LMAT is practical to run in the real world by looking at **overhead**, especially latency, throughput, memory, and cost. The main message is that sequence prediction is the slowest step, but the smaller LSTM version is light enough to deploy on modest hardware, and the storage savings can outweigh the compute cost.[58] [59] [60] [61]

Latency meaning

Overhead analysis asks: how much extra time and compute does LMAT add while monitoring traces. The paper says the sequence prediction stage is the most time-consuming part, and the multi-task version is slightly slower because it performs extra work for duration prediction in addition to event prediction. [query context]

The paper also says that using the reduced event subset, discard-top10, cuts inference time by 41 percent. The reason is that shorter input sequences and fewer possible output events make prediction cheaper. [query context]

Throughput

The paper reports that with batch size 16, the reduced LSTM processes a batch in about 19.3 ms, which corresponds to roughly 830 requests per second. Since their workload is about 1000 requests per second, this untuned setup cannot fully keep up with the incoming stream. [query context]

That means:

- Model speed is close to the workload rate, but still below it. [query context]
- So deployment may need optimization, lighter monitoring coverage, or slightly different hardware. [query context]

The paper lists three ways to improve this:

- Reduced precision and larger batches. [query context]
- Sampling only a percentage of requests. [query context]
- Choosing an appropriate CPU or low-cost GPU tier. [query context]

Practical hardware

The paper emphasizes that LMAT does not require a high-end cluster. It says the LSTM variant has about 1 million parameters, is about 5 MB on disk, and used well under 1 GB of device memory for inference in their experiments. [query context]

That makes the model fairly small by modern standards. For comparison, NVIDIA's V100, which they used for measurements, comes in 16 GB and 32 GB memory configurations, while a much cheaper T4 has 16 GB memory and a 70 W maximum power limit, so the paper's argument that modest accelerators are enough is plausible.[59] [61] [58]

Cost-benefit idea

The paper gives a simple formula for storage savings per second: saved storage depends on requests per second R, average trace size per request S, and reduction fraction ρ. In plain language, if you process more requests, if each raw trace is larger, or if LMAT reduces more trace volume, then you save more storage. [query context]

Their worked example uses:

- $R \approx 1000$ requests per second. [query context]
- $S \approx 17.5$ KB per request from 35 GB over 2,000,000 requests. [query context]
- $\rho = 0.706$, meaning 70.6 percent trace reduction. [query context]

From that, they estimate about 32,024 GB, or about 31.3 TB, saved per month. Using example storage prices around $0.012 to $0.023 per GB-month, that gives roughly $384 to $737 in monthly storage savings. Public cloud object storage references show Standard-IA around $0.0125 per GB-month and Standard storage around $0.023 per GB-month, which is consistent with the values used in the paper. [query context] [60] [62]

Break-even intuition

The break-even rule in the paper is simple: LMAT is worth it if monthly compute cost is less than monthly storage savings. Their example says a low-cost T4-class GPU could cost about $48 per month including amortized hardware and power, while the storage savings are several hundred dollars per month, so the economics favor deployment in that scenario. [query context] [61] [59]

The practical interpretation is:

- If you already have spare CPU capacity, the extra cost may be near zero. [query context]
- If you need a small GPU, the savings can still exceed the compute cost. [query context]
- The more retention, replication, backup, and indexing you do, the more valuable trace reduction becomes. [query context]

What to remember

The easiest way to read this section is:

- Latency = how long LMAT takes to process requests. [query context]
- Throughput = how many requests per second it can handle. [query context]
- Memory footprint = how much hardware is needed to run inference. [query context]
- Break-even = compare compute cost against storage savings. [query context]

So the paper's claim is not just "LMAT works," but "LMAT can be practical and cost-effective," especially with the smaller LSTM and reduced event subsets. [query context]

Would you like me to explain the cost formula step by step with units, so you can see exactly how they got the 31.3 TB per month number?

⁂

6. Discussion Scope, generality, and next steps. Our evaluation involves two systems by design: (i) the public benchmark of Fournier et al.[12], which provides kernel level traces and established scenarios; and (ii) a separate physical machine where we recreate a similar client–server stack (Apache2+PHP+MySQL) and the same tracing pipeline. The second setup extends the anomaly set and validates portability across different hardware/OS configurations while holding the workload paradigm constant, thereby, to some extent, demon strating the generality of our approach across heterogeneous platforms. More broadly, our claims of generality rest on where LMAT operates and what it models, not on HTTP-specific semantics. LMAT runs at the ker nel event layer (syscalls, scheduling, file/network I/O) and jointly forecasts event identity and duration; the control loop reacts to prediction disagree ment at this layer. These signals are application-agnostic: databases, stream processors, and microservices ultimately manifest as characteristic mixes of the same kernel primitives (e.g., send/recv, read/write, fsync, mmap, futex, context switches). Our evaluation setup is therefore not merely "an other web server": by including a relational database (MySQL) it exercises database I/O, process/thread interleavings, and network backends in addi tion to HTTP handling. That said, we acknowledge the need to empirically 6Example: PNY NVIDIA Tesla T4 16 GB GDDR6, listed at US$1,465–1,500 on Ama zon, May 2025. 7Comparable small GPUs or

consumer-grade cards yield similar costs; cloud pricing varies. 35 broaden coverage; in future work we will (a) evaluate write-heavy database workloads, (b) add streaming pipelines exhibiting back-pressure, and (c) as sess containerized microservices with synchronized, multi-host traces. Be cause LMAT's modeling and control are anchored at the kernel bound ary, these extensions should require only origin metadata (e.g., host id, container id) and no application-specific changes. Single-host vs. distributed settings. LMAT is instantiated and evaluated over kernel-level event streams, which we model as a single, time-ordered sequence. This assumption holds naturally on one host, but does not preclude multi host deployments: commodity tracing stacks (e.g., LTTng with Trace Com pass or Babeltrace) support clock synchronization and post-hoc alignment, enabling construction of a partially ordered global trace. In such settings, LMATcanoperate (i) per host with host-local control and (ii) optionally over a merged event stream where host id (and, if needed, proc id/thread id) are encoded in the event representation. Our experiments already span two distinct systems (different hardware/OS environments), and the mod eling/tracing assumptions fit both. Multi-threading and interleavings. The current setup merges events from con current processes/threads; LMAT's encoder carries process/thread identifiers so the model learns regularities amid interleavings rather than requiring a strict total order. Because events

from parallel activities are routinely re ordered in our workloads, the model is already exposed to permutations; we f ind it remains tolerant to ordering drift while flagging meaningful deviations. Extending to multi-host traces follows the same principle: adding a host id feature to the token and duration context so the model can condition on origin without losing sensitivity to cross-host temporal dynamics. Generalization and reliance on failure history. LMAT is unsupervised for detection: it learns a behavioral prior and uses prediction disagreement as a change signal. No history of failures or root-cause labels is required to function. When historical incidents exist, LMAT leverages the resulting error vectors for faster diagnosis; when they do not, LMAT clusters error vectors online so operators can quickly triage, label, and build that history over time. 36 Intermittent and rare errors. LMAT classifies each request (hundreds of ker nel events) using an online, request-level disagreement score that accumu lates over the request; escalation can occur mid-request once the rolling score crosses a threshold, so a one-off fault does not need to repeat to be detected. Our evaluation includes variable-length disturbances and low-frequency in jections; detection is not contingent on repetition, although repeated occur rences naturally improve clustering quality for diagnosis. In our current pro totype without auxiliary guardrails (no pre-trigger buffering, no post-trigger linger), the anomalous-event miss rate is already low (≈ 3%) while still yield ing substantial volume reduction. In practice, this

risk can be reduced further with lightweight engineering additions—e.g., a per-request prelude ring buffer (flush the last K events or W seconds on trigger) and a short post-trigger linger window —which do not change LMAT's detection logic but capture additional context around single, non-repeating faults and further decrease the chance of missing rare anomalies under non-continuous recording. Operator workload and novelty feedback. LMAT is designed so that opera tors label patterns, instead of individual outliers. At runtime we de-duplicate alerts by clustering request-level error vectors (Section 3); the operator labels a cluster once (e.g., "benign config change" or "expected traffic surge"), and that label propagates to all members and future near-duplicates. In practice, changes that trigger novelty—software rollouts, configuration toggles, hard ware swaps, or load regime shifts—arrive in coarse-grained batches and recur across hosts; once a benign novelty is labeled, the same pattern rarely needs re-labeling. Consequently, labeling demand decreases over time: the catalog of accepted novelties grows, and LMAT suppresses or auto-accepts matching clusters. To avoid frequent retraining, LMAT separates quick configuration ad justments from model updates. Small distribution shifts can be absorbed without full retraining by (i) duration-bin recalibration and short stabil ity windows around planned changes, and (ii) incremental updates to the LSTM/Transformer using a low learning rate. Periodic housekeeping (e.g., nightly or weekly) compacts the buffer and

revalidates thresholds but does not require offline retraining. This keeps operator effort bounded while al lowing the notion of “normal” to evolve continuously with the system. 37 6.1. Threats to Validity This subsection discusses potential threats to the validity of our study, structured around the conventional categories of internal, external, construct, and conclusion validity. Internal Validity. One internal threat is the potential bias arising from threshold selection in our adaptive tracing approach. Although thresholds for triggering detailed tracing and change detection were carefully selected using validation data to maximize F1-score, these choices might not generalize op timally to all operational contexts or unforeseen scenarios. We mitigated this threat by repeating experiments with multiple random seeds and reporting averaged results with standard deviations. External Validity. Our experiments are based on kernel trace data collected from an Apache2 web server under controlled scenarios. Although we augmented this dataset with additional noise types to improve general ization, the framework’s effectiveness may vary across different applications, workloads, or system architectures not evaluated here. The selected noisy conditions might not encompass all real-world anomaly patterns, potentially affecting LMAT’s general applicability. Future work should validate LMAT on a broader range of systems and workloads to further establish its gener alizability. Construct Validity. LMAT relies on cross-entropy loss and classifi cation accuracy metrics for

evaluating predictive performance and anomaly detection capabilities. While widely used, these metrics may not fully cap ture the practical effectiveness of adaptive tracing in real-world scenarios. Additionally, the discretization of continuous duration data into categorical bins introduces information loss and may affect sensitivity to subtle timing variations. To address this, we employed a percentile-based binning strat egy designed to retain critical distribution characteristics, but alternative duration modeling methods may yield different sensitivities. Conclusion Validity. Statistical conclusion validity refers to whether our experimental results justify the conclusions we draw from them. While experiments were carefully designed and repeated to enhance reliability, the sample size, particularly in the diversity of scenarios evaluated, could still limit the robustness of our conclusions. Ensuring statistical robustness through larger-scale validation and diverse workload experiments could further strengthen the confidence in LMAT's reported performance improvements. Addition ally, the potential for overfitting exists despite mitigation efforts through validation-based threshold selection and rigorous experimental protocols. 38 7. Conclusion and Future work This paper presented LMAT, a language model–based adaptive trac ing framework designed to support intelligent DevOps pipelines requiring both high reliability and sustainability. By dynamically focusing on the most informative segments of system traces,

including rare and non-repeating anomalies, LMAT guides operators toward periods of interest, highlights mis predicted events, and links them to potential fault sources. In doing so, it significantly reduces tracing overhead, manual analysis effort, and energy consumption. Acentral component of LMAT's effectiveness is its use of language models for system behavior modeling, with robust change detection serving as a core capability. While prior work has leveraged language models to detect anomalies in event sequences [11, 12], LMAT advances the state of the art by explicitly incorporating event durations, a critical and underutilized signal in system diagnostics. Duration modeling enables LMAT to detect behavioral changes that may not be apparent from event sequences alone, particularly in contexts where timing irregularities are indicative of system anomalies. Extensive experiments demonstrate that LMAT consistently outperforms baseline approaches across change detection, root cause analysis, and trace volume reduction. These results confirm the framework's generalizability across a diverse set of system behaviors and noise patterns. One current limitation of LMAT is its reliance on a static model that must be updated as the system's notion of "normal" evolves. For exam ple, recurring patterns initially flagged as anomalies—or one-off anomalies later deemed benign—may become part of the system's baseline behav ior. While LMAT incorporates a duration interval adjustment mechanism (see Section 4.2.2) to adapt to minor distribution shifts, broader

> behavioral changes still necessitate model updates. As shown in Figure 1, these reclassi f ied behaviors are routed through the Online Learning module, which refines the model parameters using administrator feedback or automated labeling of these behaviors. Future work will extend LMAT to multi-host and distributed tracing deployments with synchronized clocks and optional retroactive capture, and will incorporate online learning techniques [39] to enable continuous model adaptation in CI/CD environments. This enhancement will allow LMAT to preserve detection accuracy over time while maintaining its lightweight and energy-efficient profile. Periodic model refinement will ensure that the 39 system can recognize and incorporate new "normal" behaviors and prevent tracing costs from creeping back up to the level of traditional methods.

This section is the paper's "what this means, where it works, and what its limits are" part. In simple words, the authors are arguing that LMAT is not just for one exact web app setup, but they also admit that they still need more experiments on other systems and workloads.

Scope and generality

The authors try to support generality in two ways: they test on a public benchmark and on a separate physical machine with a similar software stack but different hardware and OS. Their argument is that LMAT works at the kernel-event level, so it watches low-level behavior like syscalls, file I/O, network I/O, scheduling, and timing, rather than depending on HTTP-specific application meaning.

That is an important idea: the model is not learning "web pages" or "SQL queries" directly. It is learning patterns in low-level system behavior, which many applications share, such as read, write, send, recv, mmap, futex, and context-switch related behavior.

Why they think it can transfer

The paper says databases, stream processors, and microservices all eventually appear as combinations of the same kernel primitives. So the claim is: if LMAT models those low-level patterns well, then it should transfer across many application types without major application-specific redesign.

But notice the careful wording: they say this demonstrates generality only "to some extent." That means they believe the method is broadly applicable, but they are not claiming it has already been proven everywhere.

Single host and distributed systems

Right now, the paper mainly models traces as one time-ordered sequence, which fits a single host naturally. For multi-host systems, the authors say the same idea could still work if traces from different machines are synchronized and aligned, and if metadata like host ID is added.

So their idea is:

- One host: straightforward.
- Multiple hosts: still possible, but you need synchronized clocks and extra identifiers.
- The model may run per host, or on a merged trace stream.

This means distributed deployment is presented more as an extension path than as something fully validated in the current experiments.

Multi-threading and interleavings

The paper says concurrent process and thread events are merged together, and the model includes process and thread IDs so it can learn normal interleaving patterns. In plain language, the model is not expecting a perfectly neat single-thread order; it is already trained to tolerate normal mixing of events from parallel activity.

That matters because real systems are messy. If the method only worked on perfectly ordered traces, it would be much less useful in practice.

Failure history and rare errors

A strong point in the discussion is that LMAT does not need past failure labels to detect anomalies. Detection is unsupervised, meaning it learns normal behavior and flags disagreement, while historical labeled failures help only later for faster diagnosis.

The authors also say detection does not require anomalies to repeat many times. Even one request can be flagged if its disagreement score becomes large enough, although repeated anomalies make clustering and diagnosis easier.

Operator workflow

The paper tries to reduce human effort by having operators label clusters of similar anomalies instead of one alert at a time. Once a cluster is labeled as something benign or expected, similar future cases can be auto-grouped and handled with much less manual effort.

This is an important practical point: they want the system to become easier to operate over time, not harder. The catalog of known benign or known problematic patterns is supposed to grow and reduce repeated manual work.

Model updates

The authors admit that LMAT is not fully self-maintaining yet. Minor timing drift can be handled by duration-bin recalibration, but larger changes in what "normal" means still require model updates.

So the limitation is:

- Small changes: can often be absorbed.
- Bigger behavior shifts: need updating or online learning.
- Future work: make this adaptation more continuous and automatic.

This is one of the most honest and important parts of the section, because real systems change over time.

Threats to validity

The paper lists four common research validity concerns:

- Internal validity: threshold choices may work well in their experiments but may not be best everywhere.
- External validity: tested systems and anomalies may not represent all real-world environments.
- Construct validity: metrics like cross-entropy, accuracy, F1, and duration binning may not capture every practical aspect of adaptive tracing.
- Conclusion validity: the range of scenarios may still be too limited to support very broad claims.

This is basically the authors saying, "our results are promising, but do not over-claim." That is a normal and good sign in a research paper.

Final conclusion meaning

The conclusion says LMAT's main contribution is adding explicit duration modeling to language-model-based trace analysis. Their claim is that this extra timing signal helps detect anomalies that event order alone can miss, while also reducing trace volume and supporting root-cause analysis.

So the simplest summary is:

- Old idea: model event sequences.
- New improvement: also model event durations.
- Benefit: better change detection, better diagnosis, and less tracing overhead.

The main future direction is to make LMAT work better in distributed systems and adapt more continuously as system behavior evolves.

Here is the easiest exam-style interpretation:

- Contribution: joint modeling of event type and duration.
- Why important: timing anomalies may not change event order.
- Claimed strength: lower tracing cost with strong anomaly detection.
- Main limitation: generalization and long-term adaptation still need broader validation.
- Future work: distributed traces, online learning, continuous updating.

Would you like me to make a final full-paper cheat sheet next, section by section, in very simple words?

1. https://www.machinelearningmastery.com/basics-mathematical-notation-machine-learning/
2. https://www.sciencedirect.com/topics/computer-science/evaluation-metric
3. https://www.sciencedirect.com/topics/computer-science/performance-metric
4. https://phillipi.github.io/6.s898/materials/notation.pdf
5. https://www.gaohongnan.com/notations/machine_learning.html
6. https://pmc.ncbi.nlm.nih.gov/articles/PMC10937649/
7. https://www.labelf.ai/blog/what-is-accuracy-precision-recall-and-f1-score
8. https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/
9. https://www.youtube.com/watch?v=_U6CO8JvwpE
10. https://towardsdatascience.com/understanding-evaluation-metrics-in-classification-modeling-6cc197950f01/
11. https://arxiv.org/html/2507.03392v1
12. https://human-centered.ai/wordpress/wp-content/uploads/2016/02/HOLZINGER-Machine-Learning-Notation.pdf
13. https://wiki.pathmind.com/accuracy-precision-recall-f1
14. https://www.linkedin.com/pulse/model-evaluation-metrics-data-science-mohamed-chizari-if9ye
15. https://statmodeller.com/machine-learning-evaluation-metrics-a-complete-guide/
16. image.jpg
17. image.jpg
18. https://aclanthology.org/2021.naacl-main.313.pdf
19. https://feedforward.github.io/blog/multi-task-learning-using-uncertainty/
20. https://arxiv.org/html/2408.07985v1

21. https://eurekastatistics.com/using-the-median-absolute-deviation-to-find-outliers/
22. https://en.wikipedia.org/wiki/Median_absolute_deviation
23. https://www.emergentmind.com/topics/anomaly-score-learning
24. https://arxiv.org/html/2411.14515v1
25. https://www.linkedin.com/pulse/median-absolute-deviation-mad-robust-measure-rajathilagar-r-raj--tygpf
26. https://openaccess.thecvf.com/content_cvpr_2018/papers/Kendall_Multi-Task_Learning_Using_CVPR_2018_paper.pdf
27. https://codefinity.com/courses/v2/165dbadd-b48e-4a7f-8b0d-1b8477c22a1d/047e166d-bc62-4bd1-8114-f0771ef62d83/2e168ca6-063c-4a24-9873-59915e1f4b0d
28. https://www.mathworks.com/help/vision/ref/percentilenormalizer.normalize.html
29. https://github.com/oscarkey/multitask-learning
30. https://discuss.pytorch.org/t/multi-task-learning-weight-selection-for-combining-loss-functions/152332
31. https://stackoverflow.com/questions/79255816/how-to-implement-self-paced-multitask-weighted-loss-kendall-et-al-2018-in-pyt
32. https://stackoverflow.com/questions/65403978/is-there-a-better-way-to-calculate-loss-for-multi-task-dnn-modeling
33. image.jpg
34. https://encord.com/blog/train-val-test-split/
35. https://mlu-explain.github.io/train-test-validation/
36. https://github.com/giltene/wrk2/blob/master/README.md
37. https://github.com/AmpereTravis/wrk2-aarch64
38. https://www.yangyang.cloud/blog/2018/11/05/http-load-testing-with-wrk2/
39. https://learn.microsoft.com/en-us/azure/governance/policy/samples/cis-linux/ubuntu-ado
40. https://cylab.be/blog/150/http-benchmarking-with-wrk2
41. https://objectstorage.us-ashburn-1.oraclecloud.com/p/jKUHPmZOdshGaLyHuw9GJGsLSRcNzYSo5QtrD8cSkie64pqYKOjejzN3-WX-Wo6O/n/idnreg0ywhtg/b/medium/o/CIS_Ubuntu_Linux_22.04_LTS_Benchmark_v2.0.0.pdf
42. https://httpd.apache.org/docs/2.4/programs/ab.html
43. https://ubuntu.com/engage/cis-hardening-benchmarks-usg-ubuntu2204
44. https://testdev.tools/wrk2/
45. https://github.com/giltene/wrk2
46. https://github.com/zyqCSL/wrk2
47. https://oneuptime.com/blog/post/2026-01-07-ubuntu-network-performance-tuning/view
48. https://www.v7labs.com/blog/train-validation-test-set
49. image.jpg
50. image-2.jpg
51. image.jpg

52. image.jpg
53. image-2.jpg
54. image.jpg
55. image-2.jpg
56. image.jpg
57. image.jpg
58. https://images.nvidia.com/content/technologies/volta/pdf/tesla-volta-v100-datasheet-letter-fnl-web.pdf
59. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-product-brief.pdf
60. https://www.finout.io/blog/cloud-storage-pricing-comparison
61. https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-product-literature/T4 Product Brief.pdf
62. https://cloudchipr.com/blog/amazon-s3-pricing-explained
63. https://cvw.cac.cornell.edu/gpu-architecture/gpu-example-tesla-v100/v100_mem_nvlink2
64. https://www.itcreations.com/nvidia-gpu/nvidia-tesla-t4-gpu
65. https://www.itcreations.com/nvidia-gpu/nvidia-tesla-v100-16gb-gpu
66. https://www.oracle.com/ca-en/cloud/storage/pricing/
67. https://www.fluence.network/blog/nvidia-t4/
68. https://techdocs.akamai.com/cloud-computing/docs/object-storage-pricing
69. https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf
70. https://www.leadtek.com/eng/products/ai_hpc(37)/tesla_v100(10793)/detail
71. https://www.nvidia.com/en-au/data-center/v100/
72. https://images.nvidia.com/content/technologies/volta/pdf/volta-v100-datasheet-update-us-1165301-r5.pdf